Aircheck

Brand Ad Placement Verifier

Technology Stack, Libraries, AI Model, and Working Methodology

1. Introduction

Aircheck is an AI-powered application developed to verify whether a specific advertisement has been aired within a broadcast video. Instead of manually watching hours of footage, the system automatically compares an advertisement clip with the broadcast video and identifies every occurrence of the advertisement. It also provides the exact timestamps, confidence scores, similarity graphs, and a professional PDF verification report.

The application combines **Artificial Intelligence, Computer Vision, Deep Learning, and Web Technologies** to automate the advertisement verification process.

2. Technology Stack

2.1 Python

Python is the primary programming language used for developing the entire application.

It is responsible for:

Python was selected because it provides excellent libraries for Artificial Intelligence and Computer Vision.

2.2 FastAPI

FastAPI is used as the backend framework.

It creates REST APIs that allow communication between the frontend and the AI model.

The backend performs the following operations:

FastAPI is lightweight, fast, and capable of handling multiple requests efficiently.

3. Libraries Used

3.1 OpenCV

Library: ***OpenCV (cv2)***

Purpose:

OpenCV is used for reading video files and extracting individual frames from videos. Since deep learning models cannot directly process video files, the broadcast video and advertisement clip are first divided into individual image frames.

Example workflow:

Broadcast Video â†’ Frame 1 â†’ Frame 2 â†’ Frame 3 â†’ ...

These extracted frames become the input for the AI model.

3.2 PyTorch

Library: ***PyTorch***

Purpose:

PyTorch is the deep learning framework used to load and execute the EfficientNet-B0 neural network.

Its responsibilities include:

PyTorch performs all deep learning computations in the application.

3.3 Torchvision

Library: ***Torchvision***

Purpose:

Torchvision provides pretrained EfficientNet-B0 and image preprocessing functions. Before an image is sent into the neural network, Torchvision performs preprocessing.

The preprocessing pipeline includes:

Workflow:

Video Frame â†’ Resize (224Ã—224) â†’ Convert to Tensor â†’ Normalize â†’ EfficientNet

3.4 Pillow (PIL)

Purpose:

OpenCV stores images in NumPy format. However, Torchvision expects images in PIL format. Pillow converts OpenCV frames into PIL images before preprocessing.

Workflow:

OpenCV Frame â†’ PIL Image â†’ Tensor â†’ Model

3.5 NumPy

Purpose:

NumPy performs mathematical operations required for comparing video frames. It is used for:

3.6 SciPy

Purpose:

SciPy detects peaks in similarity scores. After the system compares every frame of the advertisement with the broadcast video, similarity scores are produced. SciPy identifies the highest peaks, which represent advertisement occurrences.

3.7 Matplotlib

Purpose:

Matplotlib generates graphical reports. It creates:

These charts are also included in the final PDF report.

3.8 ReportLab

Purpose:

ReportLab generates the professional PDF verification report. The report contains:

3.9 Threading

Purpose:

Video processing requires considerable time. Python Threading allows the verification process to execute in the background while the user interface remains responsive. This enables the progress bar to update continuously during processing.

4. Artificial Intelligence Model

4.1 EfficientNet-B0

The application uses **EfficientNet-B0**, a pretrained Convolutional Neural Network (CNN).

EfficientNet-B0 was trained on the **ImageNet dataset**, which contains millions of labeled images.

Instead of using the model for image classification, this project uses it as a **feature extractor**.

5. What is a Feature Extractor?

A feature extractor converts an image into a numerical representation called an **embedding**.

Instead of identifying the object directly, the model extracts the important visual characteristics of the image.

Example:

Video Frame â†’ EfficientNet-B0 â†’ Embedding â†’ [0.42, 0.11, 0.85, ...]

Frames with similar visual content produce similar embeddings.

6. Working of the AI Model

Step 1: Upload Videos

The user uploads:

Step 2: Frame Extraction

OpenCV extracts frames from both videos.

Broadcast â†’ Frame 1 â†’ Frame 2 â†’ Frame 3 â†’ ...

Step 3: Image Preprocessing

Torchvision preprocesses each frame. Operations include:

Step 4: Feature Extraction

Each preprocessed frame is passed through EfficientNet-B0. Instead of classification, only the feature extraction layers are used. Each frame becomes an embedding vector.

Frame â†’ EfficientNet â†’ Embedding

Step 5: Cosine Similarity

The embeddings of the advertisement are compared with the embeddings of the broadcast using **Cosine Similarity**.

Cosine Similarity measures how similar two embeddings are.

Values close to 1 indicate highly similar frames.

Values close to 0 indicate different frames.

Step 6: Sliding Window Matching

The advertisement is searched throughout the broadcast using a **Sliding Window**.

The window moves across the broadcast frame by frame.

Example:

Broadcast: [A B C D E F G H I J]

Advertisement: [D E F]

Comparisons: ABC â†’ BCD â†’ CDE â†’ DEF (Highest Match) â†’ EFG â†’ FGH

The window with the highest similarity score is identified as the advertisement location.

Step 7: Peak Detection

The similarity scores are converted into a graph. SciPy detects significant peaks in this graph. Each peak represents one detected advertisement occurrence.

Step 8: Detection Information

For every detected occurrence, the system calculates:

Step 9: Report Generation

Finally, the system generates:

The report can be downloaded directly from the application.

7. Why EfficientNet-B0?

EfficientNet-B0 was selected because it provides an effective balance between performance and computational efficiency.

Its advantages include:

These characteristics make it suitable for frame comparison and advertisement verification.

8. Overall System Workflow

9. Conclusion

The Aircheck system integrates **Artificial Intelligence**, **Deep Learning**, **Computer Vision**, and **Web Technologies** to automate advertisement verification. By extracting visual features with **EfficientNet-B0**, comparing them using **Cosine Similarity** and a **Sliding Window** approach, and detecting significant matches through **peak detection**, the system accurately identifies advertisement occurrences within broadcast videos. The generated timestamps, confidence scores, visual analysis charts, and downloadable PDF reports provide a complete and reliable verification solution, reducing manual effort and improving efficiency.